

onDestroy 方法在 Android 中的使用与注意事项

下面结合代码示例，详细讲解 `onDestroy()` 的调用场景和实际用法，帮助你更直观地理解其作用。

一、基础用法：在 Activity 中重写 onDestroy ()

`onDestroy()` 是 `Activity` 类的生命周期方法，开发者可以重写它来执行销毁前的清理操作（如释放资源、取消任务等）。

```
class MainActivity : AppCompatActivity() {
    private var backgroundTask: Timer? = null // 模拟一个后台任务
    private var broadcastReceiver: MyReceiver? = null // 模拟一个广播接收器
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // 初始化后台任务（例如：定时刷新数据）
        backgroundTask = Timer().apply {
            scheduleAtFixedRate(object : TimerTask() {
                override fun run() {
                    Log.d("MainActivity", "后台任务执行中...")
                }
            }, 0, 1000) // 每1秒执行一次
        }

        // 注册广播接收器
        broadcastReceiver = MyReceiver()
        val intentFilter = IntentFilter(Intent.ACTION_SCREEN_OFF)
        registerReceiver(broadcastReceiver, intentFilter)
    }
    // 重写 onDestroy() 进行资源清理
    override fun onDestroy() {
        super.onDestroy() // 必须先调用父类方法

        // 1. 取消后台任务，避免内存泄漏
        backgroundTask?.cancel()
        backgroundTask = null

        // 2. 注销广播接收器，避免上下文泄漏
    }
}
```

```
        broadcastReceiver?.let {  
            unregisterReceiver(it)  
        }  
        broadcastReceiver = null  
  
        Log.d("MainActivity", "onDestroy() 被调用，组件即将销毁")  
    }  
    // 自定义广播接收器（示例）  
    class MyReceiver : BroadcastReceiver() {  
        override fun onReceive(context: Context?, intent: Intent?) {  
            Log.d("MainActivity", "接收到屏幕关闭广播")  
        }  
    }  
}
```

二、触发 onDestroy () 的常见场景（附代码验证）

场景 1：主动调用 finish () 销毁 Activity

当用户触发“退出”操作时，开发者调用 `finish()` 会主动销毁当前 Activity，此时 `onDestroy()` 会被调用。

```
// 在 MainActivity 中添加一个按钮点击事件，主动销毁 Activity  
findViewById<Button>(R.id.btn_finish).setOnClickListener {  
    finish() // 调用后，Activity 会进入销毁流程，最终触发 onDestroy()  
}
```

日志输出：

点击按钮后，日志会打印 `onDestroy()` 被调用，组件即将销毁，同时后台任务和广播接收器被正确清理。

场景 2：系统因配置变化销毁旧 Activity（如屏幕旋转）

当设备旋转时，系统会销毁当前 Activity 并重建新实例，旧实例的 `onDestroy()` 会被调用。

验证步骤：

1. 运行 App，确保 `MainActivity` 处于前台。

2. 旋转屏幕（需在 AndroidManifest 中不设置 `android:configChanges` 拦截配置变化，默认会重建）。

日志输出：

旧实例会打印 `onDestroy()` 被调用，随后新实例的 `onCreate()` 会被触发。

场景 3：Fragment 被移除时的 `onDestroy()`

Fragment 的 `onDestroy()` 触发时机与 Activity 类似，当 Fragment 被从布局中移除或宿主 Activity 销毁时，会调用其 `onDestroy()`。

```
class MyFragment : Fragment() {
    private var fragmentTask: Timer? = null
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_my, container, false)
    }
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        // 初始化 Fragment 的后台任务
        fragmentTask = Timer().apply {
            scheduleAtFixedRate(object : TimerTask() {
                override fun run() {
                    Log.d("MyFragment", "Fragment 后台任务执行中...")
                }
            }, 0, 1000)
        }
    }
    override fun onDestroy() {
        super.onDestroy()
        // 清理 Fragment 的资源
        fragmentTask?.cancel()
        fragmentTask = null
        Log.d("MyFragment", "Fragment 的 onDestroy() 被调用")
    }
}
```

触发方式：

在宿主 Activity 中通过 `FragmentManager` 移除 `Fragment`：

```
supportFragmentManager.beginTransaction()
    .remove(myFragment) // 移除 Fragment，触发其 onDestroy()
    .commit()
```

三、关键注意事项（代码中需规避的问题）

1. 不要依赖 `onDestroy()` 做数据持久化

系统可能在进程被强制杀死时跳过 `onDestroy()`（如内存极度不足），因此重要数据应在 `onPause()` 或 `onSaveInstanceState()` 中保存：

```
override fun onPause() {
    super.onPause()
    // 保存关键数据到 SharedPreferences 或数据库
    saveDataToLocal()
}
```

2. 避免在 `onDestroy()` 中执行耗时操作

`onDestroy()` 是组件销毁的最后一步，耗时操作可能导致组件销毁延迟，甚至引发 ANR（应用无响应）：

```
override fun onDestroy() {
    super.onDestroy()
    // 错误示例：不要在这里执行耗时操作（如下载、大量计算）
    // heavyOperation() // 禁止！可能导致 ANR
}
```

3. 确保资源完全释放，避免内存泄漏

未清理的监听器、后台线程、广播接收器等会导致组件实例无法被 GC 回收，必须在 `onDestroy()` 中彻底释放（如上述代码中的 `Timer` 和 `BroadcastReceiver`）。

总结

`onDestroy()` 是组件生命周期的“收尾方法”，主要用于释放资源，其调用时机包括：主动销毁、系统因配置变化重建、内存不足回收。实际开发中，需重点关注资源清理，避免内存泄漏，但不要依赖它执行关键业务逻辑。

（注：文档部分内容可能由 AI 生成）